

Welcome to Slidev

Presentation slides for developers

Press Space for next page →

What is Slidev?

Slidev is a slides maker and presenter designed for developers, consist of the following features

-  **Text-based** - focus on the content with Markdown, and then style them later
-  **Themable** - theme can be shared and used with npm packages
-  **Developer Friendly** - code highlighting, live coding with autocompletion
-  **Interactive** - embedding Vue components to enhance your expressions
-  **Recording** - built-in recording and camera view
-  **Portable** - export into PDF, PNGs, or even a hostable SPA
-  **Hackable** - anything possible on a webpage

Read more about [Why Slidev?](#)

agenda

- 1. [Welcome to GitOps](#)
- 2. [What is Slidev?](#)
- 3. [agenda](#)
- 4. [gitops](#)
- 13

- 5. [gitops](#)
- 6. [gitops-openfaas](#)
- 7. [gitops-argo-cd](#)
- 8. [gitops-kustomize](#)

- 9. [gitops-argo-rollouts](#)
- 10. [gitops-argo-workflows](#)
- 11. [gitops-credential-management](#)
- 12. [gitops-observability](#)

gitops

What is GitOps?

The core idea of GitOps is having a Git repository that always contains declarative descriptions of the infrastructure currently desired in the production environment and an automated process to make the production environment match the described state in the repository. If you want to deploy a new application or update an existing one, you only need to update the repository - the automated process handles everything else. It's like having cruise control for managing your applications in production.

Why should I use GitOps?

- Deploy Faster More Often
- Easy and Fast Error Recovery
- Easier Credential Management
- Self-documenting Deployments
- Shared Knowledge in Teams

How does GitOps work?

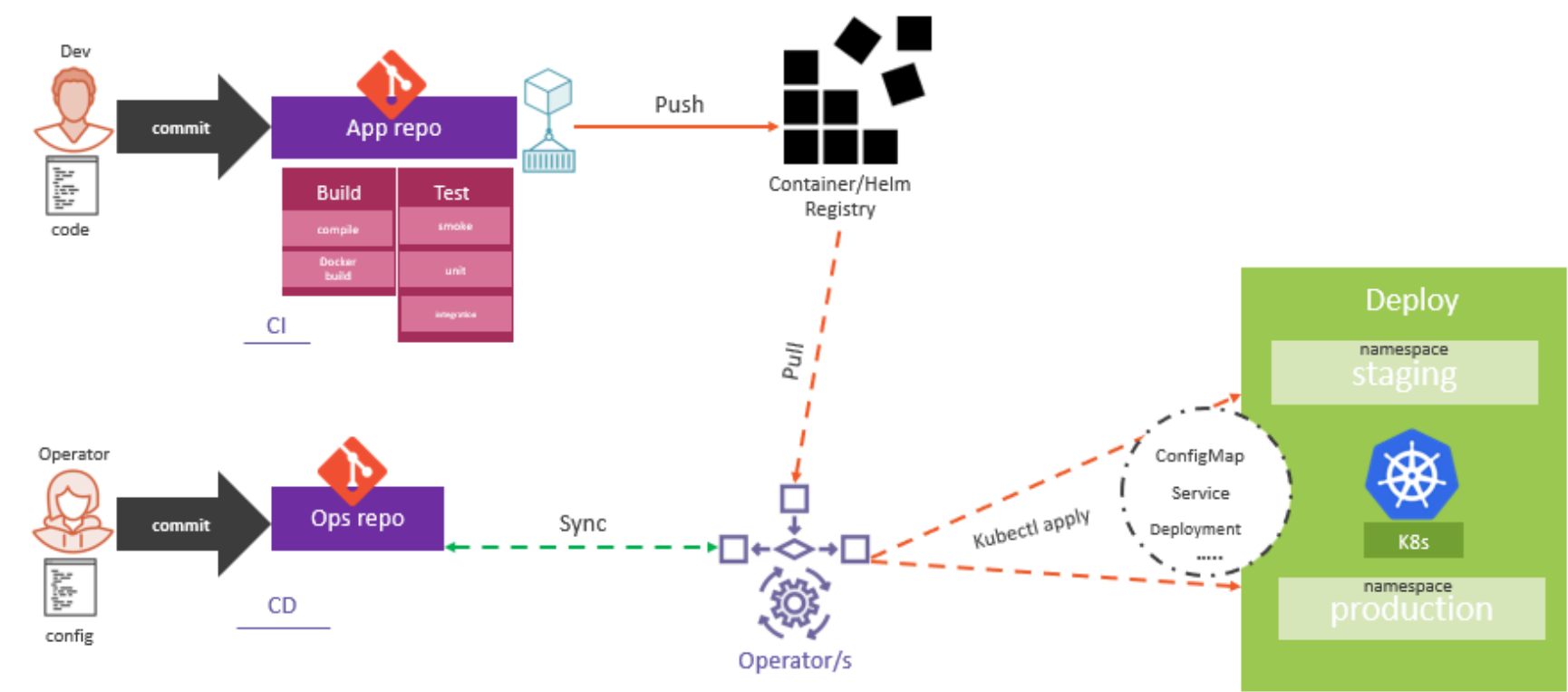
- Environment Configurations as Git repository
- Push-based vs. Pull-based Deployments
 - Push-based Deployments
 - Pull-based Deployments
- Working with Multiple Applications and Environments

GitOps Principles

- Declarative - A system managed by GitOps must have its desired state expressed declaratively.
- Versioned and Immutable - Desired state is stored in a way that enforces immutability, versioning and retains a complete version history.
- Pulled Automatically - Software agents automatically pull the desired state declarations from the source.
- Continuously Reconciled - Software agents continuously observe actual system state and attempt to apply the desired state.

gitops

How GitOps works



GitOps Principles

- **Declarative**
A system managed by GitOps must have its desired state expressed declaratively.
- **Versioned and Immutable**
Desired state is stored in a way that enforces immutability, versioning and retains a complete version history.
- **Pulled Automatically**
Software agents automatically pull the desired state declarations from the source.
- **Continuously Reconciled**
Software agents continuously observe actual system state and attempt to apply the desired state.

What you need to get started with GitOps

For you to get started with GitOps, you will need the following:

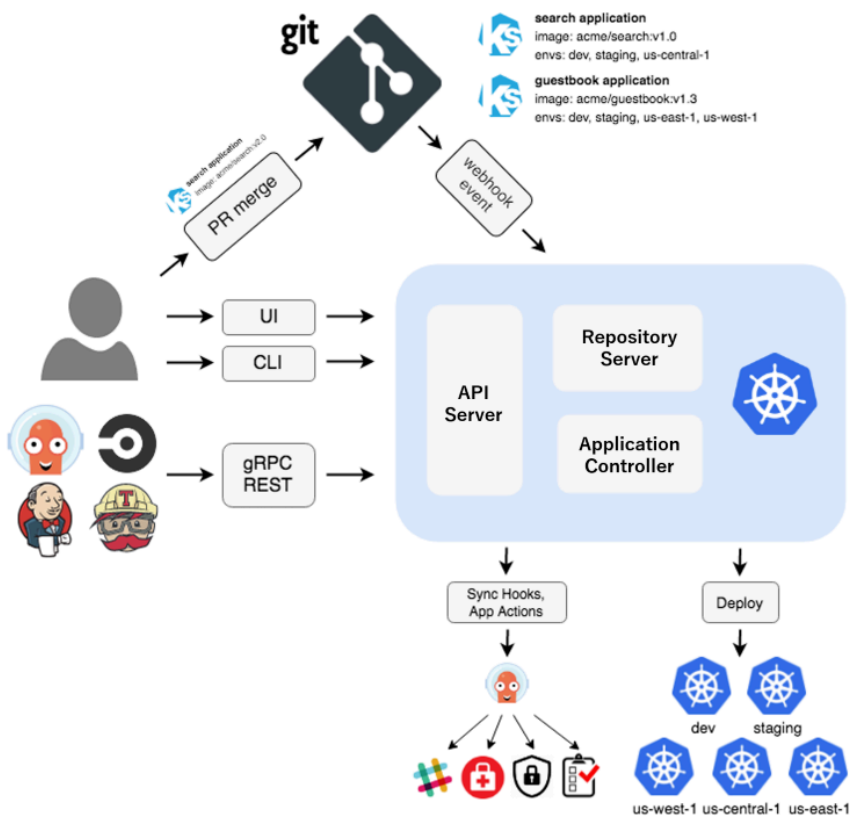
- A **Git repository** that contains your code and configuration files for your software system.
- A **GitOps Operator** or agent that can monitor your repository and synchronize your software system with it.
- A **Kubernetes cluster** or **another platform** such as cloud that can run your infrastructure and or applications.

GitOps tools and frameworks

- **Kubernetes** sits at the heart of GitOps.
- **Flux** A GitOps operator.
- **Argo CD** A GitOps operator provides declarative continuous delivery for Kubernetes.
- **OpenFaas** is a powerful framework that simplifies the deployment of functions and code.
- **Terraform** is a popular infrastructure automation tool...
- **Sonatype** is one of the top full-spectrum software supply chain management platforms.
- **Fleet** is a lightweight open-source device management tool for GitOps.
- **Helm** is one of the most robust tools for configuring Kubernetes resources.
- **Prometheus** acts as a monitoring tool for GitOps.
- **Quay** . Container images are managed meticulously with this tool.
- **Sealed Secrets** provide maximum security to your GitOps pipeline.

gitops-argo-cd

Overview



Commands

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
kubectl port-forward svc/argocd-server -n argocd 8080:443

argocd admin initial-password -n argocd
argocd login <ARGOCD_SERVER>

kubectl version --client
Client Version: v1.31.2
Kustomize Version: v5.4.2

argocd app create statements-services --repo ... --revision master --path ...
--dest-namespace ... --dest-server ... --kustomize-image ....
argocd app sync statements-services
```

Concepts

- Application A group of Kubernetes resources (CRD) as defined by a manifest.
- Application source type Which Tool is used to build the application.
- Target state The desired state of an application, as represented by files in a Git repository.
- Live state The live state of that application. What pods etc are deployed.
- Sync status Whether or not the live state matches the target state.
- Sync The process of making an application move to its target state.
- Sync operation status Whether or not a sync succeeded.
- Refresh Compare the latest code in Git with the live state. Figure out what is different.
- Health The health of the application, is it running correctly? Can it serve requests?

Manifests

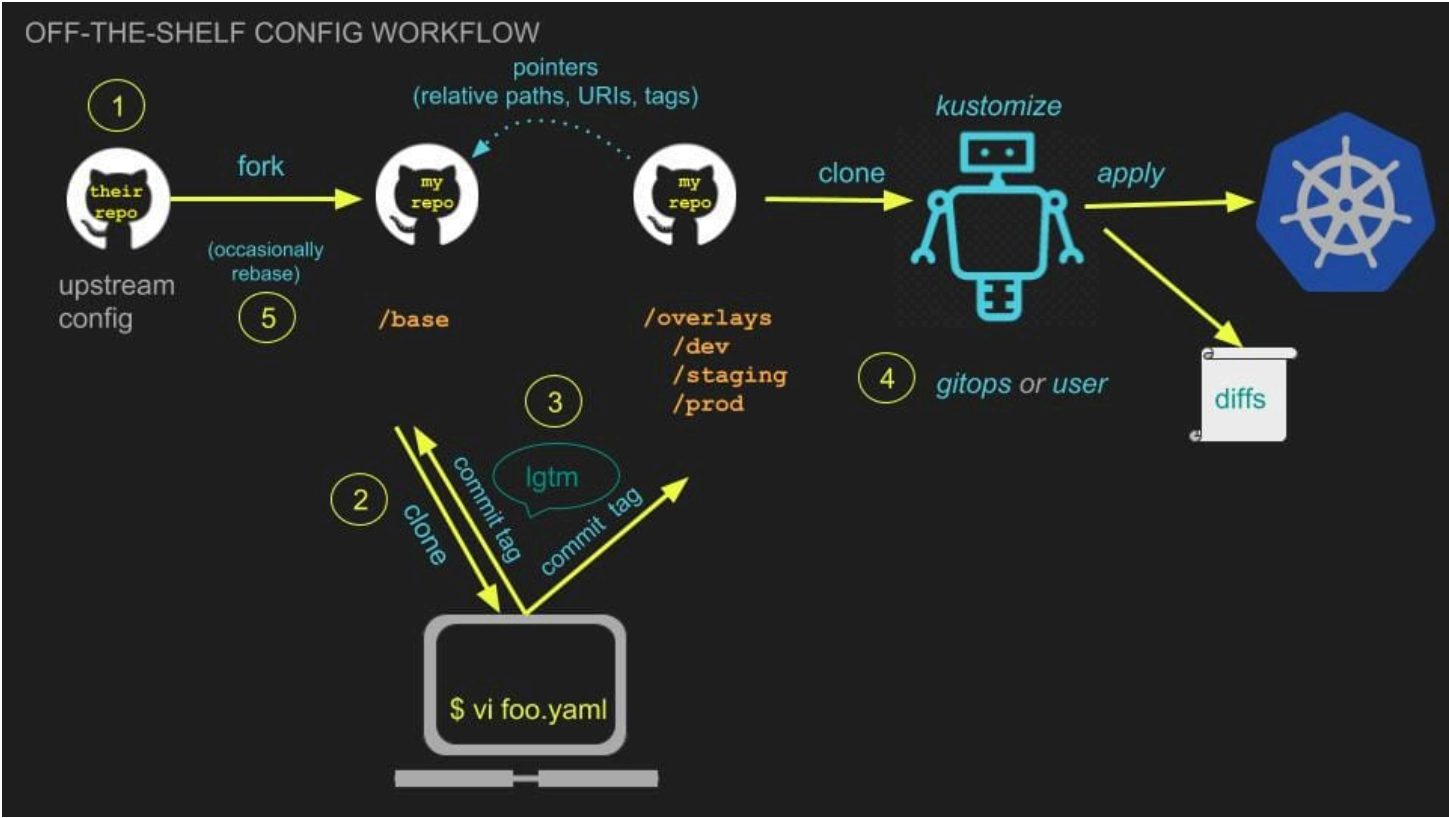
Argo CD supports several different ways in which Kubernetes manifests can be defined:

- Kustomize applications
- Helm charts
- A directory of YAML/JSON/Jsonnet manifests, including Jsonnet.
- Any custom config management tool configured as a config management plugin

Argo CD's API

gitops-kustomize

Overview



Mainfests

```
kustomize-mainfests/base$ tree
├── deployment.yaml
├── ingress.yaml
├── kustomization.yaml
├── secret.yaml
└── service.yaml

kustomize-mainfests/transactions-services$ tree
├── overlays
│   └── staging
│       ├── application.yaml
│       ├── destination-rule.yaml
│       ├── gateway.yaml
│       ├── ingress.yaml
│       ├── kustomization.yaml
│       ├── resource-quota.yaml
│       ├── staging.env
│       └── virtual-service.yaml
└── README.md
```

kustomize 术语

- kustomization kustomization 指的是 kustomization.yaml 文件，或者指的是包含 kustomization.yaml 文件的目录以及它里面引用的所有相关文件路径
- base base 指的是一个 kustomization, base 中描述了共享的内容，如资源和常见的资源配置
- overlay overlay 是一个 kustomization, 它修改(并因此依赖于)另外一个 kustomization. overlay 声明了与 base 之间的差异。通过 overlay 来维护基于 base 的不同 variants(变体)
- variant variant 是在集群中将 overlay 应用于 base 的结果。例如开发和生产环境都修改了一些共同 base 以创建不同的 variant。这些 variant 使用相同的总体资源，并与简单的方式变化
- resource 在 kustomize 的上下文中，resource 是描述 k8s API 对象的 YAML 或 JSON 文件的相对路径。即是指向一个声明了 kubernetes API 对象的 YAML 文件
- patch 修改文件的一般说明。文件路径，指向一个声明了 kubernetes API patch 的 YAML 文件

kustomize vs Helm

Helm的定位：Kubernetes 的包管理工具，而kustomize是 Kubernetes 原生配置管理。Helm将应用抽象成Chart来管理, 专注于应用的操作、复杂性管理等, kustomize 关注k8s API 对象的管理。

- Helm 提供应用描述文件模板(Go template)，在部署时通过字符替换方式渲染成 YAML，对应用描述文件具有侵入性。Kustomize使用原生k8s对象，无需模板参数化，无需侵入应用描述文件(YAML), overlay 选择相应patch生成最终YAML
 - Helm 专注于应用的复杂性及生命周期管理(包括 install、upgrade、rollback)，kustomize 通过管理应用的描述文件来间接管理应用
 - Helm 使用Chart来管理应用，Chart相对固定、稳定，相当于静态管理，更适合对外交付使用，而kustomize管理的是正在变更的应用，创建新的overlay将应用部署在新的环境，相当于动态管理，适合于 DevOps
 - Helm 通过Chart方式打包并管理应用版本，kustomize通过overlay方式管理应用不同的变体，通过Git来版本管理
- 总的来说，Helm 有自己一套体系来管理应用，而 kustomize 更轻量级，融入Kubernetes 的设计理念，通过原生 k8s API 对象来管理应用

gitops-argo-rollouts

gitops-argo-workflows

gitops-credential-management

gitops-observability